

CSE 815 · Machine Learning

Department of CSE · University of Chittagong



Neural Networks

Neurons, Layers, and Making Predictions by Forward Propagation

Rokan Uddin Faruqui

Professor, Department of CSE

University of Chittagong

`rufaruqui@cu.ac.bd`

Week 4 · Part 1

Module 1 gave us

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b), \quad g(z) = \frac{1}{1 + e^{-z}}$$

a classifier whose decision boundary is a line — unless we hand-craft features such as x_1^2 , x_1x_2 to bend it.

Hand-crafting features does not scale to an image with a million pixels. Module 2 asks: can the model *learn* its own features? A neural network does exactly that, and every one of its units is the logistic regression you already know.

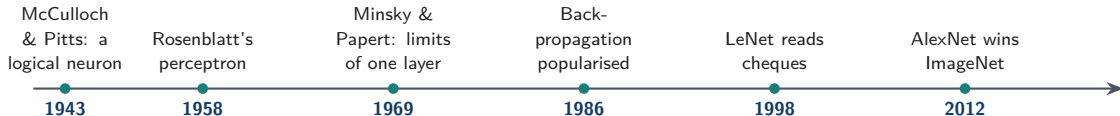
Today

1. Neurons and the brain — and where the analogy stops
2. Demand prediction: a network as stacked logistic units
3. Recognising images: features learned layer by layer
4. Inside a layer: notation and computation
5. Deeper networks and forward propagation

Part 2 turns all of this into code: TensorFlow, then NumPy from scratch, then matrix multiplication.

Neurons and the Brain

A short history



Original motivation

Write software that mimics how the brain learns. Interest rose, fell, and rose again — twice.

Today's motivation

Engineering, not biology: networks are simply the most accurate known models for speech, images and text, once there is enough data.

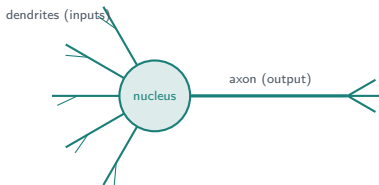
Where they won first

1. Speech recognition
2. Computer vision
3. Natural-language processing

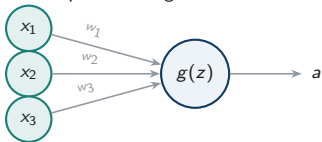
and now climate, medicine, advertising, recommendation . . .

The biological neuron, and the artificial one

Biological: dendrites → cell body → axon



Artificial: inputs → weighted sum → activation



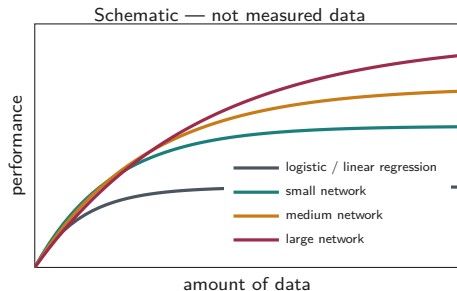
Artificial neuron (unit)

$$z = \mathbf{w} \cdot \mathbf{x} + b, \quad a = g(z)$$

a is the unit's **activation** — the number it sends on. With the sigmoid g , one unit *is* logistic regression.

A loose analogy

We still do not know how the brain learns. Modern networks owe their success to mathematics and data, not to fidelity to neurobiology. Do not argue from biology when designing a model.



Two things changed

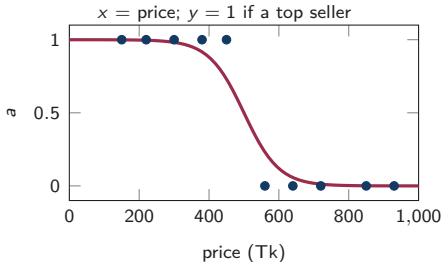
- **Data.** Digitised records, phones and the web made labelled data abundant.
- **Compute.** GPUs made the matrix arithmetic of large networks cheap — the subject of Part 2.

Key idea

Classical models plateau as data grows. Larger networks keep improving — so, given the data, a bigger network is often the most reliable route to better accuracy.

Demand Prediction

One neuron: will this T-shirt be a top seller?



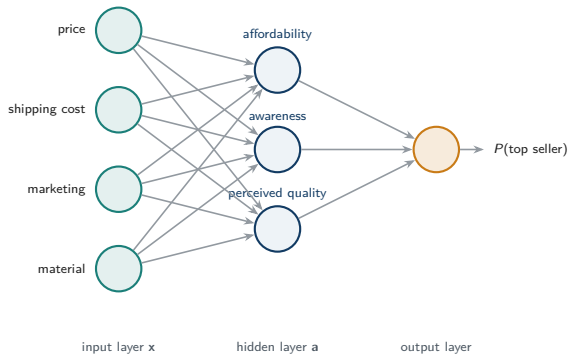
$$a = f(x) = \frac{1}{1 + e^{-(wx+b)}}$$



Vocabulary

The circle is a **neuron**. Its output a is its **activation** — here the estimated probability of being a top seller. The term comes from biology: how strongly a neuron “fires”.

Four features, three intermediate ideas, one answer



How to read it

We *imagine* that the hidden units estimate affordability (from price and shipping), awareness (from marketing), and perceived quality (from price and material). The output unit is a logistic regression on those three numbers.

Full connectivity

In practice every hidden unit sees *every* input. We do not wire affordability to price by hand — training decides which inputs each unit should use, by setting the weights.

Week 2: feature engineering

You decided the useful feature was $\text{area} = \text{frontage} \times \text{depth}$, computed it, and handed it to linear regression.

Week 4: feature learning

The hidden layer computes features $\mathbf{a}$ from $\mathbf{x}$; the output layer is a logistic regression on $\mathbf{a}$. What the features *are* is chosen by gradient descent, not by you.



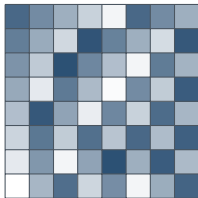
Architecture

The choices you make *before* training: how many hidden layers, and how many units in each. Weights and biases are what training then learns.

Terminology.

- **Input layer:** the feature vector $\mathbf{x}$ — it computes nothing.
- **Hidden layer:** its correct values never appear in the training set, which contains only $(\mathbf{x}, y)$.
- **Output layer:** produces the prediction.
- A network with several hidden layers is a **multilayer perceptron**.

Example: Recognising Images



each cell: brightness 0–255

A 1000×1000 greyscale photograph is a vector

$$\mathbf{x} \in \mathbb{R}^{1\,000\,000}.$$

A colour photo is three times that.

Face recognition

Input: the pixel vector $\mathbf{x}$. Output: the identity of the person.

Logistic regression on raw pixels fails — no single pixel says anything about identity, and no human can engineer a million-pixel feature set.

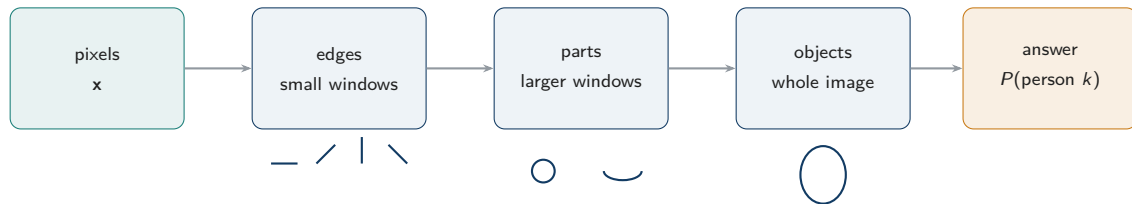
What a trained network turns out to compute

Inspecting a trained face network, researchers find:

1. first hidden layer: short **edges** at various angles;
2. second: **parts** — an eye, the corner of a nose;
3. third: coarse **whole-face** shapes.

Nobody told it to look for edges. Each layer builds on the one before.

Increasingly abstract features, learned from data



Same architecture, different data

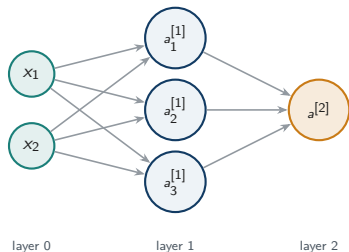
Train the identical network on photographs of cars and the hidden layers learn wheels and windows instead of eyes and noses. The architecture is generic; the features come from the data.

Interpretations are post hoc

"Layer 2 detects eyes" is a description we attach after training, by visualising what excites each unit. It is not a guarantee, and many units have no tidy label.

Inside a Neural Network Layer

Layer 1, unit by unit



Each hidden unit j has its own parameters $\mathbf{w}_j^{[1]}$, $b_j^{[1]}$, and is a logistic regression on $\mathbf{x}$.

$$a_1^{[1]} = g(\mathbf{w}_1^{[1]} \cdot \mathbf{x} + b_1^{[1]})$$

$$a_2^{[1]} = g(\mathbf{w}_2^{[1]} \cdot \mathbf{x} + b_2^{[1]})$$

$$a_3^{[1]} = g(\mathbf{w}_3^{[1]} \cdot \mathbf{x} + b_3^{[1]})$$

$$\mathbf{a}^{[1]} = [a_1^{[1]}, a_2^{[1]}, a_3^{[1]}]$$

The output layer then does the same thing to $\mathbf{a}^{[1]}$:

$$a^{[2]} = g(\mathbf{w}_1^{[2]} \cdot \mathbf{a}^{[1]} + b_1^{[2]})$$

Optional final step: predict $\hat{y} = 1$ if $a^{[2]} \geq 0.5$, else $\hat{y} = 0$ — exactly as in Week 3.

Layer notation

- $[l]$ — a quantity belonging to **layer** l .
- (i) — belonging to **training example** i (as before).
- subscript j — **unit** j within the layer.

So $a_3^{[2](7)}$ is the activation of unit 3 of layer 2 on example 7.

By convention $\mathbf{a}^{[0]} = \mathbf{x}$, so the input is “layer 0”. A network is said to have L layers when it has $L - 1$ hidden layers and one output layer: the input layer is not counted.

The one equation of this lecture

$$a_j^{[l]} = g(\mathbf{w}_j^{[l]} \cdot \mathbf{a}^{[l-1]} + b_j^{[l]})$$

Unit j of layer l takes the *whole* activation vector of the previous layer, forms a weighted sum, adds its bias, and applies the activation function g .

For now g is the sigmoid. Week 5 introduces other choices of g ; in this context g is called the **activation function**.

How many parameters does a layer have?

Counting

A layer with n_{in} inputs and n_{out} units has

$$\underbrace{n_{\text{in}} \times n_{\text{out}}}_{\text{weights}} + \underbrace{n_{\text{out}}}_{\text{biases}}$$

parameters: each unit needs one weight per input, plus its own bias.

Demand network, $4 \rightarrow 3 \rightarrow 1$:

layer 1: $4 \cdot 3 + 3 = 15$; layer 2: $3 \cdot 1 + 1 = 4$; total 19.

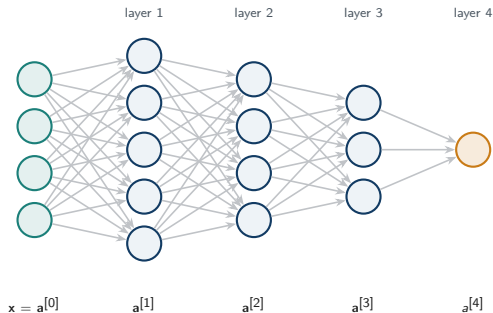
Count the inputs of *this* layer

Layer 2's units see the 3 activations of layer 1, not the 4 original features. A frequent exam error is to use n (the number of features) for every layer.

Parameter count grows with the *product* of adjacent layer widths — which is why a fully connected first layer on a million-pixel image is expensive, and why vision networks use other layer types (Week 5).

More Complex Networks and Forward Propagation

A deeper network



Four layers

Three hidden layers and one output layer — the input is not counted.

Zoom into layer 3, unit 2

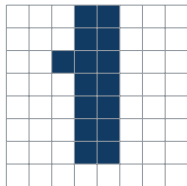
$$a_2^{[3]} = g(w_2^{[3]} \cdot a^{[2]} + b_2^{[3]})$$

$w_2^{[3]}$ has 4 entries, because layer 2 has 4 units.

Key idea

Every layer takes a vector in and gives a vector out. Stacking layers is composing functions: $f(x) = g^{[4]}(g^{[3]}(g^{[2]}(g^{[1]}(x))))$.

Handwritten digits: 0 or 1?



an 8×8 "1": $\mathbf{x} \in \mathbb{R}^{64}$



Forward propagation

$$\mathbf{a}^{[1]} = g(\mathbf{w}_j^{[1]} \cdot \mathbf{x} + b_j^{[1]})_{j=1..25} \quad 25 \text{ numbers}$$

$$\mathbf{a}^{[2]} = g(\mathbf{w}_j^{[2]} \cdot \mathbf{a}^{[1]} + b_j^{[2]})_{j=1..15} \quad 15 \text{ numbers}$$

$$a^{[3]} = g(\mathbf{w}_1^{[3]} \cdot \mathbf{a}^{[2]} + b_1^{[3]}) \quad 1 \text{ number}$$

$f(\mathbf{x}) = a^{[3]}$; predict "1" if $a^{[3]} \geq 0.5$.

Inference

Computing activations left to right, input to output, is **forward propagation**. It is all that *inference* — prediction with a trained network — requires. Learning the weights uses *backward* propagation (Week 5).

Parameters: $64 \cdot 25 + 25 = 1625$, $25 \cdot 15 + 15 = 390$, $15 + 1 = 16$; total 2031.

Layers usually narrow towards the output.

Forward propagation by hand: a $2 \rightarrow 3 \rightarrow 1$ network

Parameters

Layer 1:

$$\mathbf{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}, \quad \mathbf{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$$
$$b_1^{[1]} = -1, \quad b_2^{[1]} = 1, \quad b_3^{[1]} = 2$$

Layer 2:

$$\mathbf{w}_1^{[2]} = [2, -1, -3], \quad b_1^{[2]} = 0.5$$

Input: $\mathbf{x} = [1, 2]$.

unit	$z = \mathbf{w} \cdot \mathbf{a}_{\text{in}} + b$	$a = g(z)$
$a_1^{[1]}$	$1 \cdot 1 + 2 \cdot 2 - 1 = 4$	0.9820
$a_2^{[1]}$	$-3 \cdot 1 + 4 \cdot 2 + 1 = 6$	0.9975
$a_3^{[1]}$	$5 \cdot 1 - 6 \cdot 2 + 2 = -5$	0.0067
$a^{[2]}$	$2(0.9820) - 0.9975 - 3(0.0067) + 0.5$ $= 1.9640 - 0.9975 - 0.0201 + 0.5$ $= 1.4464$	0.8094

$a^{[2]} = 0.8094 \geq 0.5$, so $\hat{y} = 1$.

For $\mathbf{x} = [-1, -1]$: $\mathbf{z}^{[1]} = [-4, 0, 3]$, $\mathbf{a}^{[1]} = [0.0180, 0.5000, 0.9526]$,
 $z^{[2]} = -2.8217$, $a^{[2]} = 0.0562$, so $\hat{y} = 0$.

Sanity anchor

$g(0) = 0.5$ exactly, $g(4) \approx 0.982$, $g(-5) \approx 0.007$. Check hand calculations against these before trusting a long chain.

Wrap-up and Assessment

What to remember

- A unit computes $a = g(\mathbf{w} \cdot \mathbf{a}_{\text{in}} + b)$ — a logistic regression.
- A layer is many units sharing the same input vector; it maps a vector to a vector.
- Hidden layers compute features that training, not the designer, chooses.
- Deeper layers build more abstract features from earlier ones.
- Forward propagation: compute $\mathbf{a}^{[1]}, \mathbf{a}^{[2]}, \dots$ left to right.

Formulas

$$a_j^{[l]} = g(\mathbf{w}_j^{[l]} \cdot \mathbf{a}^{[l-1]} + b_j^{[l]}), \quad \mathbf{a}^{[0]} = \mathbf{x}$$

$$\text{\#params} = n_{\text{in}} n_{\text{out}} + n_{\text{out}}$$

Next

Part 2: the same computation in TensorFlow, then in NumPy from scratch, and finally as a single matrix multiplication.

1. Describe the artificial neuron: its inputs, parameters, computation and output. Explain why a single sigmoid unit is identical to logistic regression. [5]
2. Give two reasons neural networks became dominant after about 2010, and explain the performance-versus-data picture that motivates large networks. [5]
3. Using the demand-prediction example, explain the terms input layer, hidden layer, output layer and activation. Why is a hidden layer called “hidden”? [6]
4. Contrast feature engineering (Week 2) with feature learning. What does the designer still choose when using a neural network? [6]
5. Describe what the successive hidden layers of a face-recognition network are typically found to compute, and explain why such descriptions are post hoc. [5]
6. Explain the meaning of each index in $a_3^{[2](7)}$, and state the general activation equation for unit j of layer l . [5]

7. How many layers does a network with input x , three hidden layers and one output unit have? State the convention you use. [4]
8. A network has layer sizes $64 \rightarrow 25 \rightarrow 15 \rightarrow 1$. Compute the number of parameters in each layer and in total. Show your working. [6]
9. What is forward propagation, and why is it all that is needed for inference? What additional computation does training require? [5]
10. For the $2 \rightarrow 3 \rightarrow 1$ network on the worked-example slide, compute $a^{[2]}$ for $x = [0, 0]$ and state the prediction. [8]
11. Write the output of a three-layer network as a composition of functions, and explain why each layer can be viewed as a feature transformation for the next. [6]
12. In the worked example, $a_2^{[1]}$ is near 1 for both inputs shown. What does that suggest about how useful unit 2 is for distinguishing them? [5]

1. **Design a demand model.** A Chattogram garment exporter wants to predict whether a new design will sell above 10 000 units. Propose the input features, an architecture (number of layers and units) and the meaning you hope some hidden units will acquire. Justify why a network is better here than hand-engineered logistic regression — or argue that it is not. [12]
2. **Design for a parameter budget.** A microcontroller can store at most 5000 parameters. Design a network for 28×28 greyscale digit images (0 vs. 1) that fits, and show the count. What do you give up to fit? [10]
3. **Design a check for an inference engine.** You inherit code that claims to do forward propagation. Design three hand-computable test cases that would expose, respectively: a missing bias, the wrong previous-layer input, and a missing activation function. [12]
4. **Design an interpretation study.** A colleague claims hidden unit 7 of a trained X-ray classifier “detects fractures”. Design an experiment to support or refute the claim, and state what result would refute it. [10]

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§10.1–10.2** — single-layer neural networks, hidden units as learned features, and multilayer networks on the MNIST digits.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§5.1** — feed-forward network functions: the two-layer network written out unit by unit, and forward propagation.

Lab (CSE 816, Week 4): neurons and layers; coffee roasting in TensorFlow and in NumPy; practice lab on neural networks for binary classification.

Practice quizzes: *Neural networks intuition*; *Neural network model*.